

Documentación Técnica y Arquitectural: Algoritmo de Dijkstra en C++ (POO)

Postulación para el Equipo de Desarrollo de
Software VantTec — Tecnológico de Monterrey

1. Información General del Proyecto y Postulante

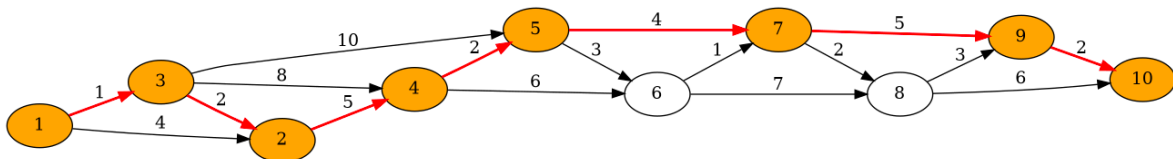
Atributo	Detalle
Nombre del Postulante	Jorge Arturo Montiel Navarro
Matrícula	A01278612
Proyecto	Dijkstra OOP Implementation for VantTec Software Developer Application
Organización Destino	Grupo Estudiantil VantTec — Tecnológico de Monterrey
Objetivo	Demostrar dominio en Programación Orientada a Objetos (POO), estructuras de datos avanzadas, optimización algorítmica e ingeniería de software modular mediante la implementación del Algoritmo de Dijkstra.

Demo :

<https://drive.google.com/file/d/1JnAlxqJBUXaBSm5MyRD-BooOt8A26px9/view?usp=sharing>

2. Resumen Ejecutivo

Este proyecto consiste en la implementación del algoritmo de Dijkstra utilizando el paradigma Orientado a Objetos (POO) en C++11. Está estructurado para solucionar el problema de la ruta más corta en grafos dirigidos y ponderados, ofreciendo una arquitectura escalable, código limpio, soporte multiplataforma y capacidades de visualización de los grafos resultantes mediante Graphviz.



3. Herramientas de Despliegue y Build System

Estándar C++11



Utilicé el estándar moderno de c++, con archivos .cpp y .hpp.

Uso de buenas prácticas:

- Separación de declaración de métodos en header file para visualización de clases de manera rápida
- Implementación en .cpp para mayor detalle

- Cada clase tiene su propio archivo, evitando problemas de dominio y búsqueda en proyectos grandes
- El código está comentado, indentado y separado correctamente. Con funciones que son reutilizadas.

CMake (CMakeLists.txt)

Fue conocimiento que aprendí en John Deere y que permite a un proyecto pesado encapsular la lógica del build y evitar pérdida de tiempo compilando diversidad cosas

```

M CMakeLists.txt
1  # Minimum CMake version this file relies on (features/syntax used below need at least this).
2  cmake_minimum_required(VERSION 3.10)
3

```

Scripts de Automatización

- [launch.bat](#) (entornos Windows) y [launch.sh](#) (entornos Linux/Unix):
Me pareció interesante crearlo para su facil despliegue y evaluación

```

launch.bat
1  # This part is to alert the user if it has installed graphviz
2  $DOT_PATH = (Get-Command dot -ErrorAction SilentlyContinue).Source
3
4  # If not found, then send an alert to the user
5  if (-not $DOT_PATH) {
6      # Send native Windows visual toast notification
7      $noti = New-Object -ComObject Wscript.Shell
8      $noti.Popup("Graphviz is not installed, install it crack", 0, "Falta Dependencia", 16) | Out-Null
9
10     Write-Error "Error: Graphviz not installed."
11     Exit 1
12 }
13
14 Write-Host "Graphviz is succesfully installed in: $DOT_PATH"
15
16 @echo off
17 where cmake >nul 2>nul
18 if %errorlevel%==0 (
19     REM Running with cmake
20     REM Clear the current terminal, specify the current directory as . and the build directory as build, then run the output
21     cls && cmake -S . -B build && cmake --build build && build\Output.exe
22 ) else (
23     cls && mkdir build && g++ main.cpp Graph.cpp Node.cpp -o build\Output.exe && build\Output.exe
24 )
25

```

4. Análisis Arquitectural y Decisiones de Diseño POO

Encapsulamiento y Abstracción

La clase principal Graph encapsula completamente la lógica interna de almacenamiento de nodos, cálculo de distancias y generación de imágenes. Los atributos clave como graph y parent se mantienen en ámbito privado (private), protegiendo la integridad de la estructura de datos ante modificaciones externas no controladas.

Estructura del Grafo

(**unordered_map<int, vector<Node>>**)

- **Justificación:** Se eligió **unordered_map** como lista de adyacencia en lugar de una matriz de adyacencia o un vector tradicional para permitir acceso y búsqueda en tiempo constante promedio $O(1)$. Esto brinda mayor flexibilidad para manejar identificadores de nodos no contiguos y asegura una alta escalabilidad en memoria sin desperdiciar espacio en matrices dispersas.

Reconstrucción de Ruta Inversa

(**unordered_map<int, int> parent**)

- **Justificación:** La reconstrucción del camino óptimo se realiza "hacia atrás". El mapa **parent** almacena la procedencia de cada nodo durante el recorrido de Dijkstra (**parent[v] = u**). Esto permite reconstruir la ruta completa desde el nodo destino hasta el origen recorriendo los padres recursivamente de forma altamente rápida y ligera en memoria.

```

unordered_map<int, vector<Node> > graph;
/*
Here we store the connection that each node has with others in the graph, using a unordered map allow us to
access in constant time and find in constant time, against a vector, finding an element is more efficient, but we
sacrifice a little bit the size of the structure used
*/

unordered_map<int,int> parent;
/*
This data structure allow us to represent the way backwards once the objective is reached, so we can mark
the way back on the image. It was decided this way because it's time efficient and we only need to acces all
the elements parents recursively until begin is reached

parent[v] = node dijkstra reached v from on the best path so far

```

5. Algoritmo de Dijkstra y Optimización de Estructuras de Datos

Cola de Prioridad con Comparador Personalizado

- **Implementación Min-Heap:** Se implementó la estructura `CompareNodeWeight` sobrecargando el `operator()` (`bool operator()(Node& n1, Node& n2) { return n1.getWeight() > n2.getWeight(); }`), lo que permite adaptar `std::priority_queue` para comportarse como un Min-Heap eficiente.
- **Optimización de Complejidad:** Al extraer iterativamente el nodo con menor distancia acumulada desde la cola de prioridad, la complejidad temporal se optimiza a $O((V + E) \log V)$, donde **V** representa el número de vértices y **E** el número de aristas.

```

struct CompareNodeWeight {
    bool operator()(Node& n1, Node& n2) {
        return n1.getWeight() > n2.getWeight(); // Min-heap
    }
};
/* This data structure is necessary because of the use of a custom of
a priority queue, allowing the comparison between nodes possible
*/

```

Flexibilidad en Tipos de Grafos

Mientras que la mayoría de implementaciones de dijkstra se centran en solo nodos que son consecutivos, esta implementación usa un mapa para que se puedan usar valores salteados.

Además no se limita a solo números, sino que es una estructura escalable que permitiría hacer el dijkstra con otros tipos de datos / structs

- El sistema soporta la definición de conexiones `(u, v, w)` representando origen, destino y peso respectivamente.
- Ofrece versatilidad al permitir tanto la inserción manual de bordes como la lectura desde archivos estáticos para grafos ponderados y no ponderados mediante funciones dedicadas (`fillWeightedGraph`, `readWeightedGraphFile`, etc.).

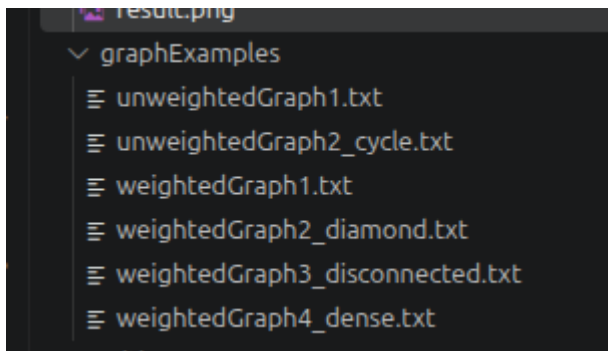
5. Visualización e Interacción con el Usuario

Generación de Diagramas (`drawGraph`)

- Exporta la topología completa del grafo junto con la ruta óptima calculada a un archivo de formato `.dot` compatible con Graphviz.
- Compila automáticamente la representación gráfica en una imagen `route.png`, facilitando la auditoría visual y la verificación rápida de los resultados del algoritmo.

Pruebas Predefinidas (`graphExercises`)

- Proporciona un menú interactivo para evaluar el algoritmo mediante casos de prueba preconfigurados o permitir la entrada de datos personalizados por parte del usuario.

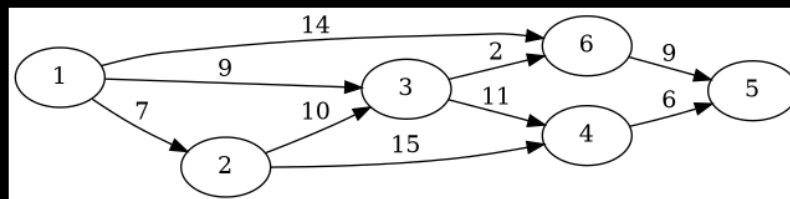


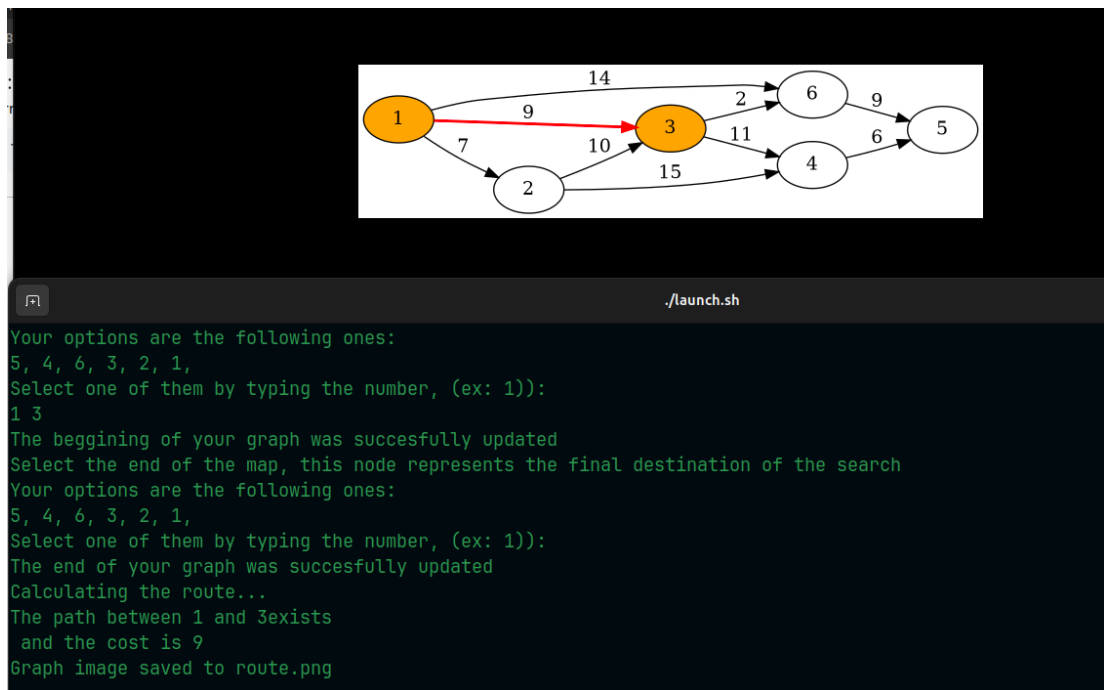
8. Pruebas

Weighted graphs

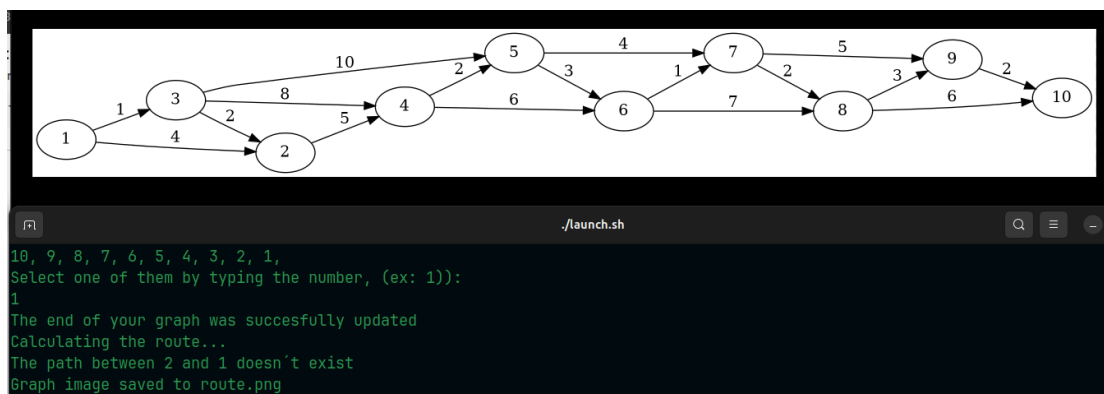
Establecí algunas pruebas preestablecidas para el testeo para grafos con pesos establecidos como las siguientes..

Para el ejemplo 1 los resultados son ...



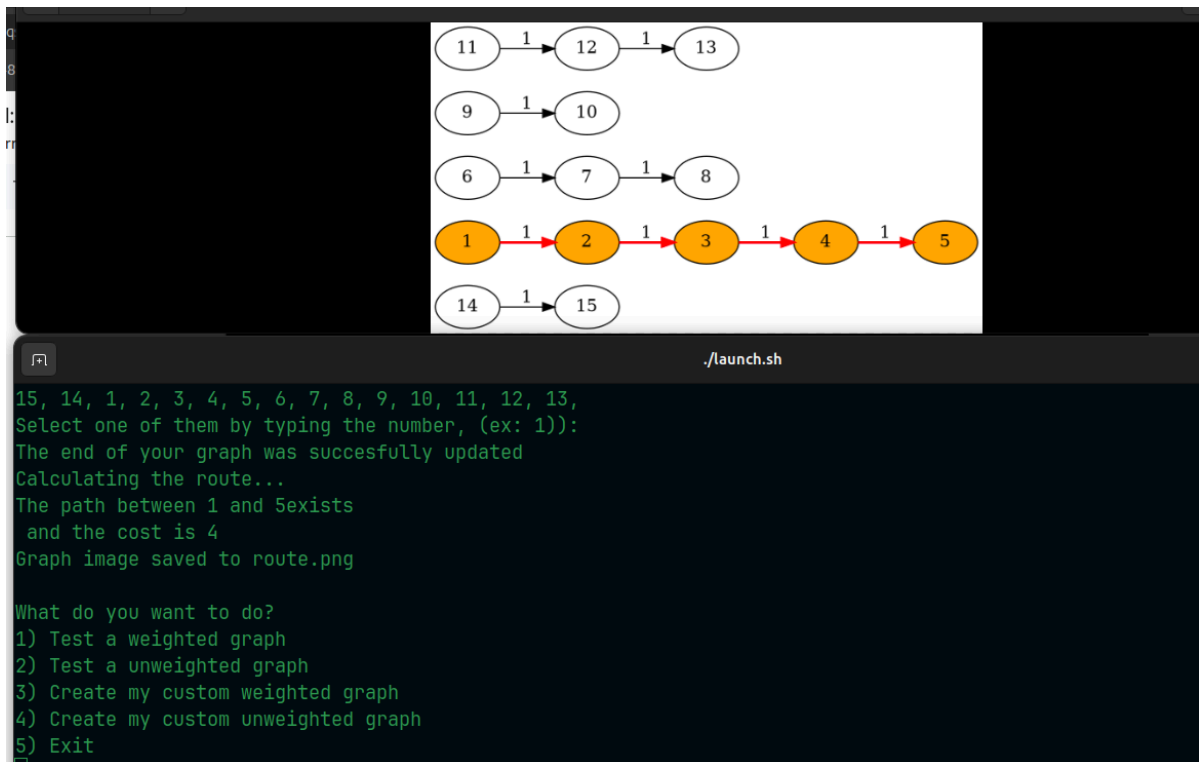


Para una ruta que no existe se tiene el siguiente resultado:



Unweighted graphs

Cuando la distancia entre dos puntos no nos es relevante un peso es interesante poder generar grafos con conexiones de peso uniforme, permitiendo obtener la distancia más corta en situaciones donde no se considera una distancia entre puntos



7. Conclusión y Valor Aportado a VantTec

Esta solución demuestra una capacidad técnica sólida en el lenguaje C++, manejo riguroso de memoria, empleo efectivo de estructuras de datos de la STL y diseño de software modular. La atención al detalle en la experiencia de usuario, la automatización de la compilación y la capacidad de visualización de datos aportan un estándar elevado compatible con los proyectos de robótica e ingeniería autónoma desarrollados en **VantTec**.

Gracias papus !!